AI Assisted Coding

Lab Assignment - 19.1

Name : B.ROHITH

HT.No: 2303A51882

Batch No: 02

Lab 19 – Code Translation: Converting Between Programming Languages

Lab Objectives:

- Understand how AI tools can assist in translating code between different programming languages.
- Learn to verify correctness and functionality after translation.
- Explore syntactic and semantic differences between languages (e.g., Python, Java, C++).
- Practice debugging and optimizing AI-translated code.

Task 1 – Python to Java Conversion

Task: Translate a simple Python class into Java using AI assistance.

Instructions:

- Prompt AI to convert a Python class with attributes and methods into equivalent Java code.
- Verify that constructors, method signatures, and data types are properly handled.
- Run both versions (Python & Java) to compare expected outputs.

Expected Output:

- Equivalent working Java class translated from Python.

Python code :

task_1.py > ...

```python
# Translate a simple Python class into Java using AI assistance.

# Python Class
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        return f"Hello, my name is {self.name} and I am {self.age} years old."
# Example usage
person = Person("Alice", 30)
print(person.greet())
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1> & "C:\Program Files\Python314\python.exe" c:/B.Tech/3-2
/AI_Assisted_Coding/lab_assignment-19.1/task_1.py
Hello, my name is Alice and I am 30 years old.
```

Converted Java Code :

task_1.java

```java
// Java Class (Translated from Python)
public class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String greet() {
        return "Hello, my name is " + name + " and I am " + age + " years old.";
    }

    public static void main(String[] args) {
        Person person = new Person("Alice", 30);
        System.out.println(person.greet());
    }
}
```

Output :

Output Generated files

```
Hello, my name is Alice and I am 30 years old.
```

Compiled and executed in **1.744 sec(s)**

Task 2 – C++ to Python Function Conversion

Task: Convert a C++ function for factorial calculation into Python.

Instructions:

- Ask AI to translate a recursive C++ factorial function into Python.
- Validate correctness by testing with sample inputs.
- Ensure Python version follows Pythonic conventions.

Expected Output:

- A correct Python implementation equivalent to the original C++ factorial function.

C++ Code :

task_2.c++

```cpp
// Convert a C++ function for factorial calculation into Python.

// C++ Code for Factorial Calculation
#include <iostream>
using namespace std;
int factorial(int n) {
    if (n == 0) {
        return 1;
    }
    return n * factorial(n - 1);
}
int main() {
    int number;
    cout << "Enter a number: ";
    cin >> number;
    cout << "Factorial of " << number << " is " << factorial(number) << endl;
    return 0;
}
```

Output :

Output

Clear

```
Enter a number: 5
Factorial of 5 is 120


=== Code Execution Successful ===
```

Converted Python Code :

```python
# Python Code for Factorial Calculation
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
# Testing the function with sample inputs
if __name__ == "__main__":
    number = int(input("Enter a number: "))
    print(f"Factorial of {number} is {factorial(number)}")
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1> & "C:\Program Files\Python314\python.exe" c:/B.Tech/3-2
/AI_Assisted_Coding/lab_assignment-19.1/task_2.py
Enter a number: 5
Factorial of 5 is 120
```

Task 3 – SQL to Pandas Query

Task: Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

SQL Code & Output :

```sql
-- Translate a SQL query into a Pandas DataFrame operation in Python.

-- Sample SQL Query:
CREATE TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    salary INT
);
INSERT INTO employees (id, name, salary) VALUES
(1, 'Alice', 60000),
(2, 'Bob', 45000),
(3, 'Charlie', 70000);
SELECT name, salary
FROM employees
WHERE salary > 50000;
```

SQL 1 / 1 1 - 0 of 0

```sql
CREATE TABLE employees (
id INT PRIMARY KEY,
name VARCHAR(50),
salary INT
);
```

SQL 1 / 1 1 - 0 of 0

```sql
INSERT INTO employees (id, name, salary) VALUES
(1, 'Alice', 60000),
(2, 'Bob', 45000),
(3, 'Charlie', 70000);
```

SQL 1 / 1 1 - 2 of 2

```sql
SELECT name, salary
FROM employees
WHERE salary > 50000;
```

name	salary
Alice	60000
Charlie	70000

Converted Python Code :

```python
# Equivalent Pandas Code:
import pandas as pd
# Sample DataFrame
data = {
    'id': [1, 2, 3],
    'name': ['Alice', 'Bob', 'Charlie'],
    'salary': [60000, 45000, 70000]
}
df = pd.DataFrame(data)
# Pandas query to retrieve name and salary where salary > 50000
result = df[df['salary'] > 50000][['name', 'salary']]
print(result)
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1> & c:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1\.venv\Scripts\Activate.ps1
(.venv) PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1> & c:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1\.venv\Scripts\python.exe c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-19.1/task_3.py
      name  salary
0    Alice   60000
2  Charlie   70000
(.venv) PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1>
```

Task 4 – Real-Time Application: Multi-Language Snippet Converter

Scenario: Students will choose a real-time use case where they need code in multiple languages.

Example:

- Converting a Python web scraping snippet into JavaScript for browser automation.
- Translating a Java method into C# for enterprise applications.

Instructions:

- Prompt AI to translate code between two chosen languages.
- Test both implementations to ensure they produce the same results.
- Document challenges faced during translation (e.g., syntax, libraries).

Expected Output:

- Working equivalent code snippets in at least two different programming languages.

Python web scraping snippet :

task_4.py > scrape_python

```python
# Students will choose a real-time use case where they need code in multiple languages.
# Example:
# • Converting a Python web scraping snippet into JavaScript for browser automation.
# • Translating a Java method into C# for enterprise applications.

from numpy import require
import requests
from bs4 import BeautifulSoup
def scrape_python(url):
    response = requests.get(url)
    soup = BeautifulSoup(response.text, 'html.parser')
    return soup.title.string
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1> & c:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1\.venv\Scripts\Activate.ps1
(.venv) PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1> & c:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1\.venv\Scripts\python.exe c:/B
.Tech/3-2/AI_Assisted_Coding/lab_assignment-19.1/task_4.py
```

Converted Javascript Code :

task_4.js > ...

```javascript
// # JavaScript equivalent using Puppeteer for browser automation
// # Note: This requires Node.js and Puppeteer installed (npm install puppeteer)
const puppeteer = require('puppeteer');

async function scrape_javascript(url) {
    const browser = await puppeteer.launch();
    const page = await browser.newPage();
    await page.goto(url);
    const title = await page.title();
    await browser.close();
    return title;
}

// Example usage (in Node.js):
// scrape_javascript('https://example.com').then(title => console.log(title));

// # Challenges faced during translation:
// # 1. Syntax Differences: Python and JavaScript have different syntax and conventions, which required
// #    careful attention to detail during translation.
// # 2. Library Differences: The libraries used for web scraping in Python (BeautifulSoup) and JavaScript (Puppeteer)
// # have different APIs and functionalities, which required understanding both libraries to ensure equivalent functionality.
// # 3. Asynchronous Programming: JavaScript's asynchronous nature with Promises and async/await required a different approach
// # compared to Python's synchronous code, which added complexity to the translation process.
```